

Bases de Dades

PRÀCTICA 4

Mario VILAR

20392724

David Díez

20392643

29 de maig de 2024

Professors: Ignasi Cos i Marc Soler



UNIVERSITAT DE
BARCELONA

ÍNDEX

1	Introducció	2
2	Exercicis	2
3	Conclusions i meme	10

INTRODUCCIÓ

La modernització de les infraestructures tecnològiques en entorns sanitaris és fonamental per garantir una gestió eficient i segura de la informació dels pacients. En aquest context en particular, el departament de *IT* d'una clínica o hospital s'enfronta al repte de reestructurar i renovar la seva base de dades, fins ara basada en SQL. Amb l'objectiu d'adoptar tecnologies més modernes i adequades a les noves necessitats, es plantegen dues opcions: la migració a una base de dades de documents, com MongoDB, o a una base de dades de grafs, com Neo4j. Aquest projecte es dividirà en diverses tasques, incloent una comparació detallada de les característiques i avantatges de cada tipus de base de dades, una decisió informada sobre la millor opció per al projecte i la creació d'un esquelet de la nova base de dades, dissenyada per ser flexible i escalable per adaptar-se a les futures necessitats de la clínica o hospital.

EXERCICIS

Exercici 2.1. *Primer de tot informeu-vos de les diferències i característiques de cada tipus de base de dades i feu-ne un resum.*

Demostració. Les dues tecnologies, MongoDB (es pot consultar la seva [pàgina web](#)) i Neo4j (un [article de BBVA](#) fa uns bons cinc cèntims), són bases de dades NoSQL molt populars. Les principals diferències entre elles són el disseny, model de dades i usos. En particular tenim

1. Model de dades:

MongoDB utilitza un model de document basat en BSON, una extensió de JSON. Cada registre en MongoDB és una col·lecció clau-valor, pel que en cada col·lecció podem trobar documents que no segueixin el mateix esquema.

Per l'altra banda Neo4j utilitza un model basat en grafs (dirigits i no dirigits, amb pesos, etc.). Les dades estan representades com nodes, arestes i propietats.

2. Flexibilitat:

A MongoDB els documents poden tenir dades jeràrquiques, matrius i estructures niuades, dades geoespacial i molt més. Amb això guanyem flexibilitat a l'hora de representar les dades, que podem «mapejar» directament com a objectes al codi.

A Neo4j les relacions estan explícitament representades i per tant la consulta i navegació és eficient. Neo4j compleix amb les propietats d'ACID (atomicitat, consistència, aïllament, durabilitat) i ofereix suport complet per a les transaccions.

3. Casos d'ús:

MongoDB s'utilitza en aplicacions web i mòbils, Big Data, IA, Edge Computing, CMS o IoT.

Neo4j té gran protagonisme en la IA generativa, també en la detecció de frau, l'anàlisi de xarxes socials, sistemes de recomanació o gestió d'accés i identitat amb dades biomètriques.

4. Consultes:

MongoDB utilitza un llenguatge de consultes basat en BSON que permet filtrar, unir, transformar i afegir documents de forma ràpida i eficient. Té la capacitat de fer consultes utilitzant JavaScript, fent que aquestes siguin enviades directament a la base de dades per ser executades.

Neo4j utilitza Cypher, un llenguatge de programació de consulta de grafs. Cypher està inspirat en SQL, amb l'addició de la coincidència de patrons prestada de SPARQL. Utilitza símbols ASCII simples per representar nodes i relacions, cosa que facilita la lectura i comprensió de les consultes. Les consultes en Cypher són naturals per trobar patrons i relacions en les dades.

5. Escalabilitat:

Gràcies al *sharding*¹, MongoDB està dissenyat per escalar horitzontalment².

D'altra banda, Neo4j és més tradicional i està enfocat en l'escalabilitat vertical. Quan augmenten les necessitats, les possibilitats d'afegir més nodes i relacions a un graf ja existent augmenten.

6. Rendiment:

MongoDB utilitza la replicació per garantir l'alta disponibilitat de les dades i el rendiment alt en operacions CRUD. Menor temps de recuperació davant fallides. Indexament personalitzat.

Les consultes de grafs, operacions de cerca i navegació en grafs a Neo4j són molt eficients, l'indexament està optimitzat. De fet, bases de dades orientades a grafs com Neo4j tenen millor rendiment que les relacionals i les no relacionals (ja que actualitzen el node i les relacions d'aquesta cerca i no tot el graf complet).

7. Desenvolupament:

MongoDB té un ecosistema de desenvolupament molt gran, destaquen MongoDB Atlas i MongoDB Compass. És un projecte en constant evolució, pel que cada versió ofereix noves millores.

Eines com Neo4j Bloom (per a interactuar visualment amb els grafs de Neo4j) són molt utilitzades per al desenvolupament de Neo4j.

De forma resumida, MongoDB es més útil per aplicacions que necessiten flexibilitat en el emmagatzematge dels documents i l'escalabilitat horitzontal mentre que Neo4j és perfecte per aplicacions on cal un us eficient de relacions i connexions complexes de les dades.

Podem agrupar totes aquestes característiques en la següent taula:

¹ El *sharding* és un tipus de partició de bases de dades que separa les grans bases de dades en parts més petites, més ràpides i més fàcils de gestionar.

² L'escalabilitat horitzontal implica dividir el conjunt de dades del sistema i la càrrega en diversos servidors, afegint servidors addicionals per augmentar la capacitat segons sigui necessari.

Característica	MongoDB	Neo4j
Model de dades	Documents BSON flexibles	Nodes, arestes i propietats
Flexibilitat	Alta, estructures variables	Alta, relacions explícites
Casos d'ús	Aplicacions web/mòbils, Big Data, CMS	Anàlisi de xarxes, sistemes de recomanació
Consultes	BSON, potent en operacions CRUD	Cypher, optimitzat per a grafs
Escalabilitat	Horitzontal amb sharding	Vertical
Rendiment	Rendiment alt en operacions CRUD	Rendiment alt en consultes de grafs.
Desenvolupament	MongoDB Atlas i MongoDB Compass	Neo4j Bloom

Taula 1: Característiques de MongoDB i Neo4j

Exercici 2.2. *Presenteu una taula amb pros i contres d'utilitzar cadascuna de les possibilitats en aquest projecte específic, així com un esbós de disseny per a cada opció.*

Demostració. En aquest apartat veurem dues taules diferents, ambdues tenen els pros i contres d'utilitzar MongoDB i Neo4j, respectivament.

MongoDB	
Pros	Contres
Flexible per a estructures de dades canviants Alta escalabilitat horitzontal Bona per a grans volums de dades no estructurades Model de dades senzill per a documents heterogenis Gran comunitat i suport	Menys eficient en gestió de relacions complexes Les agregacions grans poden ser problemàtiques Pot necessitar més espai d'emmagatzematge Sense reemplaçament per a les solucions d'herència Encara és una tecnologia jove

Taula 2: Pros i contres d'utilitzar MongoDB

Neo4j	
Pros	Contres
Millor per a gestionar relacions complexes Consultes ràpides en estructures de graf Model natural per a dominis amb moltes relacions Optimitzat per a cercar camins i patrons en grafs Bona visualització de dades gràfiques	Escalabilitat horitzontal limitada Dificultat afegida per aprendre Cypher Més costos per a consultes simples no relacionals Menys optimitzat per a dades no relacionades Menys popular i menys recursos disponibles Complicat fragmentar un graf densament connectat

Taula 3: Pros i contres d'utilitzar Neo4j

Pel que fa l'esbós del disseny a MongoDB un exemple seria

```
{
  "pacients": [
    {
```

```

4      "patient_id": "12345",
5      "patient_info": {
6          "nom": "Leo Messi",
7          "dob": "24-06-1987",
8          "address": "Gran Via",
9          "telf": "101010101"
10     },
11     "medicacions": [
12         {
13             "nom": "Vicodina",
14             "dosis": "7.5/300 mg",
15             "data_inici": "01-01-2024",
16             "data_final": "05-01-2024"
17         }
18     ],
19     "historial_medic":
20     [
21         {
22             "condicio": "Lesió femoral",
23             "data_diagnosis": "01-01-2024"
24         }
25     ],
26     "doctors": [
27         {
28             "doctor_id": "d123",
29             "nom": "Gregory House",
30             "especialitat": "Nefrologia"
31         }
32     ]
33 }
34 ]
35 }

```

on l'objecte pacient té la informació de la persona en `patient_info`, la llista de medicacions, l'historial mèdic i els doctors que l'han tractat. L'esbós pel disseny en Neo4j seria

```

1 CREATE (p: Pacient {patient_id: "12345", nom: "Leo Messi", dob: "24-06-1987", address:
   ↳ "Gran Via", telf: "101010101"})
2 CREATE (m: Medicacio {nom: "Vicodina", dosis: "7.5/300 mg", data_inici: "01-01-2024",
   ↳ data_final: "01-05-2024"})
3 CREATE (hm: Historial_medic {condicio: "Lesió femoral", data_diagnosis: "01-01-2024"})
4 CREATE (d: Doctor {doctor_id: "d123", nom: "Gregory House", especialitat: "Nefrologia"})
5 CREATE (p)-[:PREN]->(m)
6 CREATE (p)-[:PATEIX]->(hm)
7 CREATE (p)-[:TRACTAT PER]->(d)

```

On PREN és la relació entre Pacient i Medicacio. PATEIX és entre Pacient i Historial_mèdic i, per últim, TRACTAT_PER és entre Pacient i Doctor. ■

Exercici 2.3. *Preneu una decisió informada en base a la taula de l'activitat anterior sobre quina s'adapta millor al projecte. El que es demana aquí es veure el raonament i la justificació de la tria.*

Demostració. Després de veure les principals avantatges entre MongoDB i Neo4j podem concloure dos fets.

1. **Complexitat de relacions:** Si la base de dades ha de gestionar moltes relacions complexes entre pacients, medicacions, historial clínic, metges, etc, Neo4j és la millor opció, gràcies a la seva optimització per a grafs.
2. **Flexibilitat i escalabilitat:** Si el disseny requereix flexibilitat en l'estructura de dades (dades inestructurades o semiestructurades, amb esquemes en constant evolució) i escalabilitat horitzontal senzilla, MongoDB és la millor opció. En el nostre cas, això és relativament innecessari.

Vist això, en el nostre cas particular Neo4j és el més adequat pel projecte ja que les relacions entre pacients, medicacions, metges i altres són essencials per la gestió de la clínica. La capacitat de Neo4j per gestionar i optimitzar aquestes relacions fa que les consultes i la navegació entre les dades siguin més eficients i intuïtives.

Podríem haver prioritzat el creixement horitzontal, però pensem que en una clínica els doctors, consultes, medicaments, etc. canvien molt poc al llarg del temps i, per tant, no hem donat tanta importància al creixement (diguem-ne flexibilitat i escalabilitat), sinó a l'estructura. Per aquest motiu i els esmentats a l'anterior paràgraf, considerem que Neo4j és l'eina adequada. ■

Exercici 2.4. *Creeu un esquelet d'exemple de la base de dades proposada, tenint en compte que ha de poder créixer i adaptar-se a les necessitats futures de l'hospital.*

Demostració. Aquest esquelet proporciona una base inicial per a la base de dades de Neo4j, permetent que es pugui expandir fàcilment amb més nodes i relacions segons les necessitats futures del centre hospitalari. El codi i el disseny dels nodes, arestes i propietats és el següent.

```

1 // Crear pacients
2 CREATE (p1: Pacient {patient_id: "12345", nom: "Leo Messi", dob: "24-06-1987", address:
  ↳ "Gran Via", telf: "101010101"})
3 CREATE (p2: Pacient {patient_id: "54321", nom: "Cristiano Ronaldo", dob: "05-02-1985",
  ↳ address: "Petita Via", telf: "777777777"})
4
5 // Crear medicacions
6 CREATE (m1: Medicacio {nom: "Vicodina", dosis: "7.5/300 mg", data_inici: "01-01-2024",
  ↳ data_final: "01-05-2024"})
7 CREATE (m2: Medicacio {nom: "Ibuprofeno", dosis: "200mg", data_inici: "02-02-2024",
  ↳ data_final: "10-05-2024"})
8

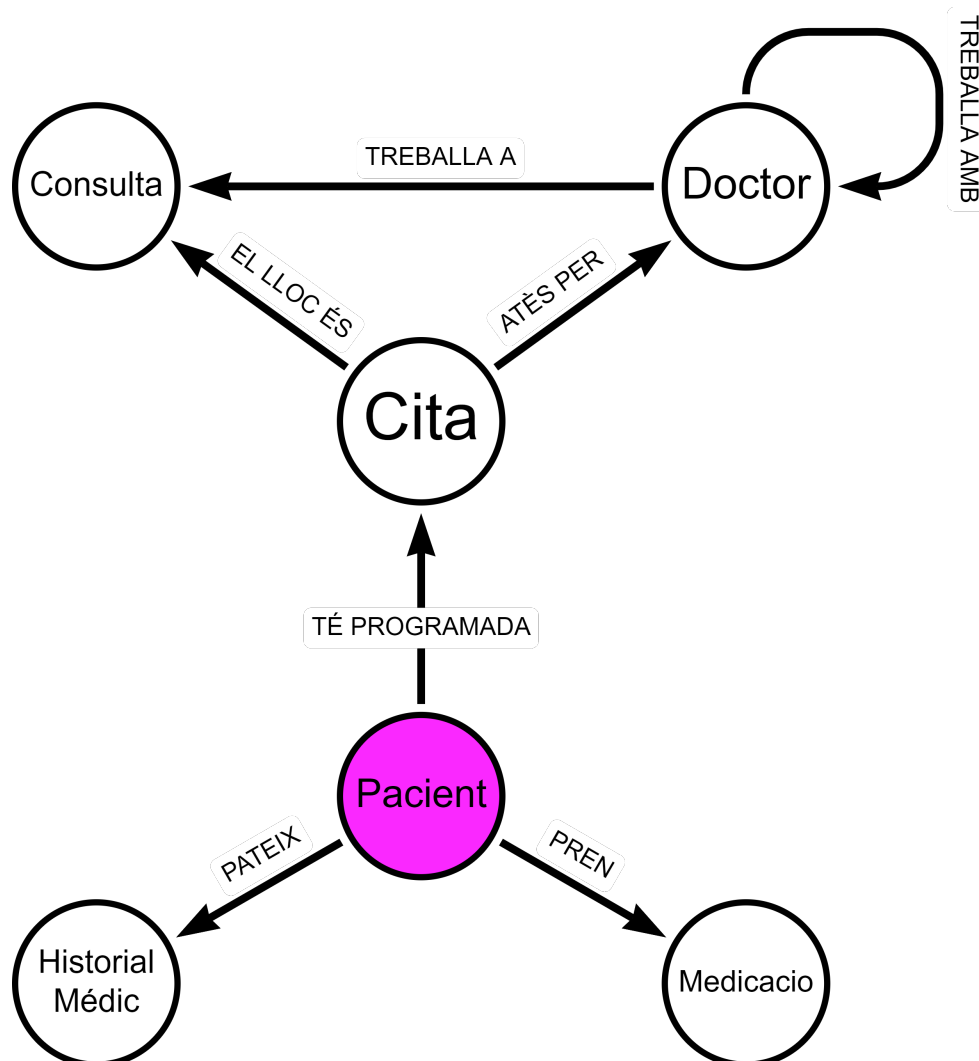
```

```

9 // Crear historial mèdic
10 CREATE (hm1: Historial_medic {condicio: "Lesió femoral", data_diagnosis: "01-01-2024"})
11 CREATE (hm2: Historial_medic {condicio: "Diabetes", data_diagnosis: "02-02-2024"})
12
13 // Crear metges
14 CREATE (d1: Doctor {doctor_id: "d123", nom: "Gregory House", especialitat: "Nefrologia"})
15 CREATE (d2: Doctor {doctor_id: "d543", nom: "Maria", especialitat: "Endocrino"})
16
17 // Crear consultes de la clínica
18 CREATE (c1: Consulta {id_consulta: "C000", nom: "1A", planta: "Primer pis", edifici: "A"})
19 CREATE (c2: Consulta {id_consulta: "C111", nom: "1B", planta: "Primer pis", edifici: "B"})
20
21 // Crear cites
22 CREATE (ci1: Cita {dia: "22-05-2024", hora: "10:00", consulta: "C000", doctor: "d123"})
23 CREATE (ci2: Cita {dia: "23-05-2024", hora: "11:30", consulta: "C111", doctor: "d543"})
24
25 // Relacions entre pacients i medicaments
26 CREATE (p1)-[:PREN]->(m1)
27 CREATE (p2)-[:PREN]->(m2)
28
29 // Relacions entre pacients i historial mèdic
30 CREATE (p1)-[:PATEIX]->(hm1)
31 CREATE (p2)-[:PATEIX]->(hm2)
32
33 // Relacions entre pacients i cites
34 CREATE (p1)-[:TÉ PROGRAMADA]->(ci1)
35 CREATE (p2)-[:TÉ PROGRAMADA]->(ci2)
36
37 // Relacions entre cites i metges
38 CREATE (ci1)-[:ATÈS PER]->(d1)
39 CREATE (ci2)-[:ATÈS PER]->(d2)
40
41 // Relacions entre cites i consultes
42 CREATE (ci1)-[:EL LLOC ES]->(c1)
43 CREATE (ci2)-[:EL LLOC ES]->(c2)
44 // Relacions entre doctors i consultes
45 CREATE (d1)-[:TREBALLA A]->(c1)
46 CREATE (d2)-[:TREBALLA A]->(c2)
47
48 // Relacions entre doctors
49 CREATE (d1)-[:TREBALLA AMB]->(d2)

```

La sintaxi no és del tot estranya als usuaris de $\text{\LaTeX}$ que usin el paquet TikZ. Com a mínim des de la perspectiva d'un servidor. No és que siguin idèntiques, però tenen clars paral·lelismes. El disseny resultant és:



El resum de les accions del codi és el següent:

- Creació de pacients: Es creen nodes que representen pacients amb informació personal com el nom, data de naixement, adreça i telèfon.
- Creació de medicaments: Es defineixen nodes per a diferents medicaments, incloent el nom, dosi i dates d'inici i final del tractament.
- Creació d'historials mèdics: Es generen nodes que contenen informació sobre condicions mèdiques dels pacients i les dates de diagnòstic.
- Creació de metges: Es creen nodes per a metges, amb informació com el nom, especialitat i identificador del metge.
- Creació de consultes: Es defineixen nodes per a les consultes de la clínica, amb detalls sobre la seva ubicació dins de l'edifici.
- Creació de cites: Es generen nodes per a les cites mèdiques, especificant la data, hora, consulta i metge assignat.

El criteri d'establiment de relacions que hem seguit és el següent:

- Entre pacients i medicaments: Per indicar quin medicament pren cada pacient.
- Entre pacients i historials mèdics: Per associar pacients amb les seves condicions mèdiques.
- Entre pacients i cites: Per programar les cites mèdiques dels pacients.
- Entre cites i metges: Per especificar quin metge atén cada cita.
- Entre cites i consultes: Per indicar on es realitza cada cita dins de la clínica.
- Entre metges i consultes: Per assenyalar en quina consulta treballa cada metge.
- Entre metges: Per mostrar que dos metges treballen junts.

Aquestes accions construeixen una estructura de dades que permet gestionar i consultar informació sobre pacients, tractaments, diagnòstics, personal mèdic, i la logística de les cites dins de la clínica.

És interessant comentar que en cas hem considerat bona idea associar els pacients amb les cites i, a partir d'allà, obtenir el doctor. Podríem haver fet un enllaç directe entre pacient i doctor però creiem que és més còmode així. En cas de fer una implementació real i trobar-nos amb un problema hauríem de fer el canvi.

Observem també que aquesta implementació és més detallada que la donada abans. La primera consistia en un petit resum de com seria i ara hem volgut donar més detall de la implementació així com un diagrama visual. ■

CONCLUSIONS I MEME

En aquest projecte, hem explorat les opcions de modernització de la base de dades d'una clínica o hospital, considerant MongoDB i Neo4j com a alternatives a SQL. Hem identificat les diferències clau entre una base de dades de documents i una de grafs, destacant avantatges com la flexibilitat i escalabilitat de MongoDB i la gestió de relacions complexes de Neo4j. També hem analitzat els inconvenients de cadascuna, incloent la menor eficiència de MongoDB en relacions complexes, com era el cas de la consulta mèdica que hem proposat, i la limitada escalabilitat horitzontal de Neo4j. Finalment, hem proporcionat esbossos de disseny per ambdues opcions, adaptant-nos a les necessitats específiques d'una institució mèdica i ajudant a prendre una decisió informada sobre quina tecnologia adoptar.



Figura 1: Últim meme de l'assignatura. :(